

Parecer Técnico — Protótipo Funcional Maria Mineira (Rails 8.1)

Contexto

O documento em anexo ("PROMPT — PROTÓTIPO FUNCIONAL DO SITE MARIA MINEIRA") descreve os requisitos funcionais, de produto e de privacidade para o primeiro protótipo do site Maria Mineira: uma plataforma de informação, orientação, conexão com serviços e "escuta digital" para mulheres em Minas Gerais, com forte componente de inteligência territorial construída a partir de sinais anônimos e estruturados da jornada do usuário. O diretório de trabalho atual está vazio (projeto greenfield, sem repositório git), então este parecer parte do zero, sem código legado a considerar.

O pedido do usuário é um **parecer técnico de arquitetura** (não a implementação em si) cobrindo viabilidade, modelagem, organização de projeto, decisões técnicas, e comparação de APIs externas com preços pesquisados em fontes atuais (agosto/2026). Este arquivo consolida esse parecer; após aprovação, ele pode ser publicado como artifact/documento e, se desejado, servir de base para o scaffold real do app Rails.

***Nota de atualização.** Após a primeira versão deste parecer, a liderança do projeto revisou o PDF original e trouxe uma correção de leitura importante sobre como as "duas entradas" da jornada devem funcionar na prática — especialmente que a coleta de motivo/localização deve acontecer **dentro da própria conversa do chatbot**, e não em telas de formulário separadas antes dele. Esta versão já incorpora esse ajuste (ver seções 1, 2 e 3, marcadas onde relevante). O modelo de dados de fundo (tags, eventos, taxonomia) não muda — o que muda é a superfície de UX/fluxo de telas.*

1. Visão geral da solução e entendimento funcional

A Maria Mineira **não é um chatbot** — é um ecossistema com três frentes que já existem (identidade visual, Instagram, mapeamento territorial de equipamentos) e uma capacidade nova a construir: transformar buscas espontâneas em **sinais digitais anônimos e estruturados**, analisáveis de forma agregada por território.

Pontos centrais extraídos do PDF:

- **Princípio de experiência:** a usuária escreve livremente o que precisa, sem conhecer categorias administrativas. A tecnologia interpreta nos bastidores.
- **Dois entradas na jornada (leitura confirmada pela liderança do projeto):**
 - **Entrada 1 — origem por conteúdo (passiva, sem conversa).** Instagram e páginas educativas do site são organizados por **tipo de violência** (ex.: violência patrimonial, violência psicológica), pela Lei Maria da Penha, por serviços específicos (delegacia

especializada de atendimento à mulher, casa de abrigo etc.). Quando a pessoa chega a partir desse conteúdo — e o rastreamento estiver disponível ("se tiver ativado ou não", nas palavras da liderança: nem todo acesso carrega uma origem rastreável, ex.: link direto sem parâmetro) — o sistema registra tag_origem/subtag_origem automaticamente. **Não há formulário nem pergunta nessa entrada.**

- **Entrada 2 — o chatbot, quando a mulher quer tirar dúvidas (ativa).** É a única porta conversacional. É **dentro do próprio chat** — não em telas de formulário isoladas antes dele — que a Maria Mineira pergunta o que a pessoa busca (gera tag_motivo/subtag_motivo) e o município/localização dela, sem pedir para repetir a história (não-revitimização). É também dentro dessa mesma conversa que o equipamento mais próximo é indicado e que se descobre, na prática, quais municípios estão "vazios" (sem equipamento cadastrado adequado) — esse cruzamento é a base do diagnóstico territorial. Tudo isso é armazenado de forma estruturada (tags + categoria + equipamento + município), não como narrativa livre.
- ***Observação de leitura:*** o PDF original descreve isso como "duas interações" e desenha telas separadas de motivo/localização/resultado antes do chatbot (seção 27/28 do PDF). A liderança esclareceu que a intenção real é mais simples: **motivo e localização são turnos de conversa dentro do chatbot**, não etapas de wizard fora dele. Este parecer já reflete essa correção (ver seções 2 e 3); vale confirmar com a equipe por escrito antes de congelar o wireframe (pendência na seção 9).
- **Três tags independentes** (tag_origem, tag_motivo, tag_chat), cada uma com subtag, permitindo observar mudança de tema ao longo da jornada — este é o dado central do produto. tag_origem continua vindo da Entrada 1 (conteúdo); tag_motivo e tag_chat agora nascem ambas dentro da Entrada 2 (chatbot), diferindo apenas pelo momento da conversa em que aparecem (primeiro assunto declarado vs. assuntos que surgem depois).
- **Taxonomia hierárquica administrada pela equipe** (não pelo time técnico), reutilizada em Instagram, site, campanhas e chatbot — com exemplos concretos já citados pela liderança: tipos de violência, Lei Maria da Penha, delegacia especializada de atendimento à mulher, casa de abrigo.
- **Geolocalização** usada operacionalmente na sessão, mas **não deve virar dado estatístico preciso** — para inteligência territorial usa-se nível agregado (município/região).
- **Municípios sem cobertura:** nunca afirmar "não existe serviço"; usar "ainda não temos informações suficientes" + registrar a procura mesmo sem resultado.
- **Chatbot não é canal de denúncia**, não deve coletar narrativas detalhadas com fins estatísticos, e **não deve armazenar conversas integralmente por padrão** — o padrão é persistir dados estruturados (tags, categoria, equipamento, município), não o texto livre.
- **Inteligência territorial futura:** cruzamento de 4 camadas (estrutura territorial, sinais Maria Mineira, dados administrativos/SEJUSP, sinais externos/notícias), sem inferência automática de causalidade e sem apresentar sinais digitais como "casos de violência" ou prova de subnotificação.
- **Mobile first**, pensado para tráfego vindo do Instagram/campanhas.

- O PDF lista 10 telas prioritárias (seção 27) e um fluxo de ponta a ponta (seção 28); com o ajuste da liderança, as telas de "motivo", "localização" e "resultado" deixam de ser páginas isoladas e passam a ser turnos dentro da tela única de chatbot — reduzindo a jornada pública, na prática, a: Homepage → Chatbot (com motivo, localização e resultado embutidos) → Mapa/conteúdo educativo como telas de apoio, sempre com CTA para voltar ao chat.

Este entendimento está bem coberto pelo PDF — é o documento mais completo e opinativo em termos de *produto* que já vi para um caso assim; as lacunas estão nos *dados e ativos concretos* (taxonomia definitiva, base de equipamentos, identidade visual, orçamento), tratadas na seção 9.

2. Proposta de arquitetura (Rails 8.1 + Importmap + Stimulus)

Recomendação: monólito Rails 8.1 com Hotwire (Turbo + Stimulus via importmap-rails), sem build step de Node/JS bundler. Justificativa: o fluxo é majoritariamente formulário → resposta → navegação (não é um SPA denso), o requisito mobile-first pede carregamento rápido (importmap evita bundling e reduz complexidade de CI/CD), e a "sofisticação nos bastidores, simplicidade na superfície" (princípio final do PDF) combina bem com um monólito bem estruturado em vez de front-end + API separados.

Pilares técnicos do Rails 8.1 a usar:

Necessidade	Solução Rails 8.1
Assets	Propshaft (padrão)
JS sem build step	importmap-rails + Stimulus (padrão)
Fila de background jobs	Solid Queue (padrão, sem Redis)
Cache	Solid Cache (padrão)
Real-time (chat, Turbo Streams)	Solid Cable (padrão, sem Redis)
Autenticação (área admin)	Gerador nativo <code>bin/rails generate authentication (has_secure_password)</code>
Jobs longos/retomáveis	Active Job Continuations (novidade 8.1) — útil para importações/geocodificação em lote
Observabilidade estruturada	Structured Event Reporting (novidade 8.1) — complementa (não substitui) a tabela de eventos de jornada, ver seção 3
Segredos	Credenciais nativas (<code>config/credentials.yml.enc</code>) para chaves de API

Separação lógica de dados (multi-DB, recurso nativo do Rails ≥6, reforçado no 8):

Duas bases distintas na mesma app, mapeando diretamente a exigência do PDF (seções 14 e 24) de separar **"dados necessários para orientação"** de **"dados necessários para estatística"**:

- **primary** — conteúdo (páginas, tipos de violência, políticas), taxonomia, equipamentos/território, parceiros, usuários administrativos.
- **analytics** (ou **signals**) — eventos de jornada (**tag_origem**, **tag_motivo**, **tag_chat**, município, resultado da busca), fisicamente isolada, com política de acesso e retenção próprias. Isso facilita também aplicar controles de acesso e retenção diferentes por banco (ex.: **analytics** pode ter TTL e menos privilégio de leitura do que **primary**).

Módulos da aplicação (bounded contexts dentro do monólito):

1. **Journey/Escuta digital** — sessão anônima, captura de origem (Entrada 1), eventos estruturados de toda a jornada.
2. **Taxonomy** — tags/subtags hierárquicas, administradas via painel.
3. **Territorial** — municípios, equipamentos/serviços, categorias de serviço, busca por proximidade.
4. **Content** — páginas educativas (tipos de violência, direitos, políticas, programas) — fonte da Entrada 1 (origem por conteúdo).
5. **Chat** — **é agora o único ponto de entrada conversacional (Entrada 2)**: recebe o **tag_origem** de contexto quando existir, conduz a conversa em estágios (saudação → motivo → localização → resultado → livre), e é o lugar onde **tag_motivo**, **tag_chat** e a busca por equipamento mais próximo acontecem — via Turbo Streams + Solid Cable, numa única tela.
6. **Partners** — Rede Maria Mineira.
7. **Admin** — CMS interno, dashboard futuro (stub de leitura de eventos).

Front-end (Stimulus controllers principais):

chat_controller (Turbo Stream, autoscroll, indicador de digitando, renderiza dentro do fluxo do chat tanto a textarea de motivo quanto os botões de localização e os cards de resultado — ver seção 3.2), **geolocation_controller** (Geolocation API do navegador + fallback manual, acionado por um botão **dentro** de uma mensagem do chat, não por uma tela separada), **map_controller** (integra MapLibre GL JS via importmap — ver seção 6/7 sobre por que não Google Maps JS), **quick_exit_controller** (ver seção 8 — recomendação de segurança fora do PDF, a validar).

Deploy: Rails 8 traz Kamal 2 como padrão para deploy via Docker em qualquer VPS. Como este ambiente já tem o MCP do **Railway** configurado, ele é uma alternativa direta (Postgres gerenciado, deploy via `railway up`, variáveis/segredos geridos) — vale confirmar com o usuário qual dos dois é o destino real (ver pendência na seção 9).

3. Models, controllers, schema e responsabilidades

3.1 Domínio "Journey" (banco analytics)

Journey::Session

```
id (uuid), started_at, tag_origem, subtag_origem,  
plataforma_origem, campanha, conteudo_origem, pagina_entrada,  
municipio_id (nullable), device_hint (mobile/desktop, sem fingerprint)
```

Journey::Event # tabela central de eventos, ver seção 15 do PDF

```
id, journey_session_id, event_type (enum: origem, motivo, busca_servico,  
resultado_busca, chatbot, pagina_conteudo), tag, subtag, municipio_id,  
categoria_servico, equipamento_id (nullable), resultado (encontrado/nao_encontrado),  
distancia_aproximada_km, created_at
```

Journey::ChatTurn # granularidade da 2ª interação

```
id, journey_session_id, tag_chat, subtag_chat, created_at  
# texto livre NÃO persistido aqui por padrão (seção 14 do PDF) –  
# ver "Chat::Message" abaixo, que vive separado, criptografado e com TTL.
```

Journey::Session e Journey::Event são o coração do requisito de "inteligência territorial": qualquer pergunta da seção 20 do PDF (o que se procura por território, mudança de tema entre chegada e chatbot etc.) é uma consulta agregada sobre Journey::Event.

3.2 Domínio "Chat" (mensagens efêmeras, banco separado ou tabela com regras próprias)

Chat::Conversation

```
id, journey_session_id, started_at,  
stage (enum: saudacao, aguardando_motivo, aguardando_localizacao,  
apresentando_resultado, livre), # ver 3.2.1 – ajuste pós-feedback  
context_tag (herdado da origem/motivo, evita repetição)
```

Chat::Message

```
id, chat_conversation_id, role (user/assistant/system_card), body_ciphertext (Active  
Record Encryption),  
card_type (nullable – enum: location_prompt, facility_results,  
municipio_sem_cobertura),  
created_at  
# job recorrente (Solid Queue) apaga body_ciphertext após retenção definida  
# (ex.: N dias) – apenas Journey::ChatTurn (tags) sobrevive indefinidamente.
```

Isso implementa literalmente a seção 14 do PDF: dado estruturado sobrevive, narrativa sensível tem vida curta e controlada.

3.2.1 Fluxo conversacional embutido (ajuste pós-feedback da liderança)

A correção trazida pela liderança muda a *superfície* do produto, não o modelo de tags. Em vez de MotivesController, LocationsController e ServiceSearchesController como telas de formulário anteriores ao chat (como eu havia desenhado na primeira versão deste parecer), a captura de motivo e localização passa a ser conduzida por um único service object dentro do próprio chat:

```

Chat::TurnHandler
  # chamado a cada mensagem enviada pela usuária dentro de
Chat::ConversationsController
  # decide o que fazer com base em conversation.stage:
  #
  # saudacao          -> envia "Como a Maria Mineira pode te ajudar hoje?"
  #                   e avança para aguardando_motivo
  # aguardando_motivo -> texto livre vira input do
Classification::MotiveClassifier
  #                   grava tag_motivo/subtag_motivo (Journey::Event),
  #                   renderiza card "usar minha localização" / "informar
manualmente"
  #                   (card_type: location_prompt), avança para
aguardando_localizacao
  # aguardando_localizacao -> recebe coordenadas (Stimulus geolocation_controller) OU
  #                           texto (município/bairro/CEP), chama
  #                           Territorial::NearestFacilityFinder, grava
resultado_busca
  #                   (Journey::Event) e renderiza card de equipamentos
  #                   (ou "ainda não temos informações suficientes...", seção
11 do PDF),
  #                   avança para apresentando_resultado
  # apresentando_resultado -> oferece continuar conversando; qualquer mensagem nova
  #                           passa a ser classificada como tag_chat/subtag_chat
  #                           (Classification::MotiveClassifier reutilizado),
  #                           avança para livre
  # livre               -> conversa aberta; cada tópico novo detectado gera
  #                           um novo Journey::ChatTurn (tag_chat/subtag_chat)

```

Isso preserva 100% dos dados exigidos pelo PDF (seção 15: tag_motivo, subtag_motivo, municipio, categoria_servico, equipamento_indicado, tag_chat) e a regra de não pedir para repetir a história (o context_tag da conversa carrega o tag_origem, se existir, para a Maria Mineira já cumprimentar de forma direcionada quando a origem for conhecida). O que muda é só a interface: tudo acontece como turnos de uma conversa, com botões/cards inline, em vez de páginas de formulário sequenciais.

3.3 Domínio "Taxonomy" (banco primary)

```

Taxonomy::Tag
  id, slug, label, kind (enum: geral, tipo_violencia), parent_id (self-referential),
  active, position

```

Usada como fonte única para tag_origem/motivo/chat, para as páginas de conteúdo e para o mapeamento tag → categoria de serviço. Estrutura em árvore simples (tag/subtag) é suficiente pelo que o PDF pede; não há indicação de precisar de mais de 2 níveis.

3.4 Domínio "Territorial" (banco primary)

```

Territorial::Municipality
  id, ibge_code, name, region, state (default "MG")
  # seed via IBGE Localidades API (grátis) – ver seção 6

```

```

Territorial::Facility (Equipamento)

```

id, name, facility_type, address, neighborhood, cep, municipality_id, latitude, longitude, phone, opening_hours, description, access_info, active

Territorial::ServiceCategory

id, taxonomy_tag_id (liga a categoria de serviço à taxonomia de motivo), name, slug

Territorial::FacilityServiceCategory # join

facility_id, service_category_id

FacilitysController#nearby (ou um service object Territorial::NearestFacilityFinder) recebe service_category + lat/lng (ou municipality_id) e retorna adequação + proximidade combinadas — implementando literalmente a regra do PDF: "o equipamento mais próximo não deve ser indicado como melhor opção se não for adequado".

3.5 Domínio "Content" (banco primary)

Content::Page

id, content_type (enum: tipo_violencia, direito, politica, programa, servico_info, campanha, informacao_util), title, slug, body (Action Text), taxonomy_tag_id, show_find_service_cta, show_chat_cta, published_at

Um único model com content_type cobre as seções 18 e 19 do PDF sem duplicar estrutura — página de "tipo de violência" é apenas um content_type específico com CTAs obrigatórios ("Encontrar serviço próximo", "Conversar com a Maria Mineira").

3.6 Domínio "Partners" (banco primary)

Partners::Partner

id, name, partner_type, description, url, logo (Active Storage), coverage_scope (texto livre ou municipality_ids), active

3.7 Admin

Admin::User

id, email, password_digest (gerador nativo Rails 8), role (enum: super_admin, content_editor, data_analyst, partner_manager)

3.8 Controllers principais (telas da seção 27 do PDF, já com o ajuste da liderança aplicado)

Tela do PDF	Controller/Rota	Observação
Homepage	HomeController#index	CTA único "Como posso te ajudar?" abre o chat
Tela de motivo	*(embutida)* Chat::MessagesController#create	via Chat::TurnHandler, estágio aguardando_motivo — não é mais uma rota própria
Tela de localização	*(embutida)* Chat::MessagesController#create	estágio aguardando_localizacao; botão de geolocalização renderizado como card do chat

Resultado	*(embutida)* Chat::MessagesController# create	estágio apresentando_resultado, card facility_results
Chatbot	Chat::ConversationsContro ller#show/create, Chat::MessagesController# create (Turbo Streams)	tela única que concentra motivo, localização e resultado
Município sem equipamento	*(embutida)* branch dentro do Chat::TurnHandler	card municipio_sem_cobertura, não é tela separada
Mapa	MapController#index, Territorial::Municipaliti esController#show, Territorial::FacilitiesCo ntroller#show	tela de apoio, navegável a partir do chat ou do menu
Tipos de violência	Content::ViolenceTypesCon troller#index/show	fonte da Entrada 1 (origem por conteúdo)
Página de tipo de violência	idem #show	CTAs "Encontrar serviço" / "Conversar com a Maria Mineira" abrem o chat carregando tag_origem
Página de políticas/direitos	Content::PagesController# index/show	idem

Namespace Admin:: espelha os mesmos domínios (Admin::TagsController, Admin::FacilitiesController, Admin::PagesController, Admin::PartnersController, Admin::DashboardController — este último apenas leitura agregada de Journey::Event, servindo de stub para o dashboard completo citado na seção 20/27 do PDF, que **não precisa** ser construído nesta fase).

3.9 Service objects (lógica de domínio fora dos controllers)

- Chat::TurnHandler — orquestra os estágios da conversa (motivo → localização → resultado → livre), descrito em 3.2.1; é o service que substitui os antigos controllers de motivo/localização/resultado.
- Classification::MotiveClassifier — recebe texto livre, devolve tag_motivo/subtag_motivo ou tag_chat/subtag_chat conforme o estágio (ver seção 5 sobre estratégia camada-regras + LLM fallback).
- Journey::EventRecorder — grava eventos de forma consistente (usado pelo Chat::TurnHandler e pelos controllers de conteúdo, para tag_origem).
- Territorial::NearestFacilityFinder — adequação + proximidade, chamado pelo Chat::TurnHandler no estágio aguardando_localizacao.
- Chat::ContextBuilder — monta o contexto inicial da conversa a partir do tag_origem (se a pessoa veio de uma página de conteúdo), evitando que a Maria Mineira comece "do zero" quando já se sabe algo sobre o que trouxe a pessoa até ali (requisito da seção 4 do PDF, agora aplicado também à origem, não só ao motivo).

4. Organização de pastas e módulos

```
app/  
  controllers/  
    admin/  
    chat/          # conversations, messages – concentra motivo/localização/resultado  
    content/  
    territorial/  
    (controllers públicos de nível superior: home, map, partners, pages)  
  models/  
    journey/  
    chat/  
    taxonomy/  
    territorial/  
    content/  
    partners/  
    admin/  
  services/  
    classification/  
    territorial/  
    journey/  
    chat/  
  jobs/  
    classify_motive_job.rb  
    purge_stale_chat_messages_job.rb  
    geocode_facility_job.rb  
    sync_municipalities_job.rb  
  javascript/  
    controllers/ (Stimulus: geolocation, motive_form, chat, map, quick_exit)  
  views/  
    (espelha controllers; partials compartilhados em views/shared)  
  channels/ (se necessário além dos helpers de Turbo Streams)  
config/  
  database.yml      # multi-db: primary + analytics  
  recurring.yml     # Solid Queue: purge job, sync municípios  
  credentials.yml.enc # chaves de API (LLM, mapas)  
db/  
  primary_migrate/  
  analytics_migrate/  
lib/  
  taxonomy/        # seeds fornecidos pela equipe (YAML), carregados via task  
test/  
  system/          # Capybara – fluxo de ponta a ponta da seção 28 do PDF  
  models/, services/, jobs/
```

Justificativa: nomear os domínios como módulos (Territorial::, Content::, Journey::) desde o início evita colisão de nomes comuns (Session, Event, Message) e comunica a separação PII-leve vs. estrutural que o PDF exige.

5. Considerações técnicas importantes

Validações. Texto livre do motivo: presença + limite de tamanho generoso (evitar frustrar a mulher com erro de validação), sem lista fechada obrigatória (requisito explícito do PDF).

Localização manual: aceitar município, bairro ou CEP — não exigir todos. Rate limiting (rack-attack) nos endpoints públicos para mitigar scraping/abuso, dado o caráter sensível do público-alvo.

Autenticação. Zero login para a usuária final — por desenho, alinhado ao anonimato pedido pelo PDF. Apenas a área Admin: : usa o gerador nativo de autenticação do Rails 8. Recomenda-se 2FA (ex. gem `rotpp`) para contas admin, dado o tema sensível dos dados geridos.

Autorização. Pundit ou Action Policy para os 4 papéis administrativos hipotetizados (`super_admin`, `content_editor`, `data_analyst`, `partner_manager`) — **os papéis exatos não estão no PDF**, é hipótese a validar (seção 9).

Background jobs (Solid Queue, nativo, sem Redis).

- `ClassifyMotiveJob` — chama o classificador (regras → LLM fallback) de forma assíncrona quando a latência do provedor externo puder impactar UX; ou síncrono se a camada de regras resolver a maioria dos casos rapidamente.
- `PurgeStaleChatMessagesJob` — recorrente, aplica a política de retenção de `Chat::Message` (seção 14 do PDF).
- `GeocodeFacilityJob` — geocodifica endereço → lat/Ing **uma vez**, no cadastro/edição do equipamento (baixíssimo volume, não por visitante). Beneficia-se de Active Job Continuations (novidade 8.1) se for rodado em lote na carga inicial da base territorial.
- `SyncMunicipalitiesJob` — sincroniza `Territorial::Municipality` com a API do IBGE.

Uploads. Active Storage apenas para o lado administrativo (logos de parceiros, mídia de páginas de conteúdo, ativos de identidade visual fornecidos pela equipe). **Nenhum upload é solicitado à usuária final no PDF** — isso é bom: reduz superfície de risco (nenhum arquivo sensível sobe do lado do público).

Integrações — detalhadas na seção 6, mas resumo de decisão: classificação de texto (LLM com fallback de regras), geocodificação/mapa (gratuita para o MVP), dados de município/CEP (gratuitos, oficiais).

Performance e escalabilidade.

- SQLite é o padrão do "Solid stack" do Rails 8 e funciona bem para o volume inicial de um protótipo, mas **buscas geoespaciais de proximidade** (equipamento mais próximo) são um ponto de atenção: recomenda-se **PostgreSQL + PostGIS** assim que o projeto sair de protótipo puro, pelas funções nativas de distância (`ST_DWithin`/`ST_Distance`) — muito mais eficiente que calcular haversine em Ruby para toda a base a cada busca. Para o protótipo

inicial, uma função SQL de haversine simples é aceitável dado o volume ainda pequeno de equipamentos.

- Cache agressivo (Solid Cache) para páginas de conteúdo educativo, que mudam pouco.
- CDN para os ativos de marca (fornecidos pela equipe) e imagens de conteúdo.

Observação de segurança de domínio (não pedida no PDF, mas relevante ao tema): sites voltados a mulheres em situação de violência costumam incluir um "botão de saída rápida" e cuidado para não deixar rastro óbvio no histórico/notificações do navegador, pois o dispositivo pode ser monitorado por um agressor. **Isso não está no PDF** — está listado aqui como recomendação a validar com a equipe (seção 8/9), não como requisito assumido.

6. APIs externas — opções, finalidade e custo

6.1 Geolocalização de município / CEP (dado territorial oficial brasileiro)

API	Finalidade	Link oficial	Preços	Observação
IBGE — API de Localidades	Lista oficial de municípios de MG com código IBGE, para popular <code>Territorial::Municipality</code> e o seletor de localização manual	https://servicodados.ibge.gov.br/api/docs/localidades	Gratuita, sem limite documentado de uso razoável	Fonte oficial, ideal para seed e refresh periódico
ViaCEP	CEP → endereço/bairro /município/código IBGE	https://viacep.com.br/	Gratuita	Simples, mantida há anos, boa cobertura
BrasilAPI	CEP, município, e outros dados brasileiros num único wrapper comunitário	https://brasilapi.com.br/	Gratuita	Projeto comunitário; evitar varreduras massivas (política de uso pede não fazer scraping de todos os CEPs)

6.2 Geocodificação de endereço → coordenadas (para os equipamentos cadastrados) e mapa interativo

API	Finalidade	Link oficial	Preços	Observação
-----	------------	--------------	--------	------------

Google Maps Platform (Geocoding + Maps JS)	Geocodificar endereços e exibir mapa	https://mapsplatform.google.com/pricing/	10.000 eventos grátis/mês (SKU Essentials), depois ~US\$5/1.000 requisições	Termos restringem cache de resultados a 30 dias sem plano especial — atrito com a exigência do PDF de reter apenas dados agregados; melhor precisão/autocomplete, mas custo cresce com escala e há resistência de ToS a armazenar coordenadas indefinidamente
Mapbox (Geocoding + GL JS)	Geocodificação + mapa interativo	https://www.mapbox.com/pricing	100.000 requisições grátis/mês (geocodificação temporária); geocodificação "permanente" (para armazenar, nosso caso) é cobrada à parte, sem tier grátis	Bom para o mapa visual; para geocodificar e *guardar* coordenadas de equipamentos, entra no plano pago desde a 1ª requisição
OpenStreetMap Nominatim (instância pública ou self-hosted)	Geocodificação de endereços dos equipamentos, feita 1x no cadastro (baixo volume)	https://nominatim.org/release-docs/latest/api/Overview/ (política de uso: https://operations.osmfoundation.org/policies/nominatim/)	Gratuita (instância pública, limite ~1 req/s, uso não comercial pesado) ou grátis + custo de infra se self-hosted	Melhor encaixe para este caso: geocodificação é feita pelo admin ao cadastrar equipamento (dezenas/centenas de vezes, não por visitante), volume baixo o suficiente para a política pública, ou trivial de self-hostar depois

MapLibre GL JS + tiles gratuitos (OpenFreeMap / MapTiler free tier)	Renderização do mapa interativo no front-end	https://maplibre.org/	Biblioteca open-source e gratuita; tiles gratuitos com limites generosos	Evita taxa de licenciamento de mapa (Google Maps JS/Mapbox GL cobram por carregamento de mapa); mantém sofisticação visual sem custo recorrente
--	--	---	--	---

6.3 Classificação de texto livre / chatbot conversacional (NLP/LLM)

API	Finalidade	Link oficial	Preços	Observação
Anthropic Claude API (Haiku 4.5)	Classificar tag_motivo/sub tag_motivo e casos de baixa confiança da camada de regras; possivelmente conduzir trechos do chatbot	https://www.anthropic.com/pricing (docs: https://docs.anthropic.com)	US\$1 / US\$5 por milhão de tokens (entrada/saída)	Modelo mais barato da Anthropic capaz de seguir instruções de classificação; textos curtos custam frações de centavo por chamada; *prompt caching* reduz ainda mais custo de instruções repetidas
Anthropic Claude API (Sonnet 5)	Conversação mais elaborada do chatbot (explicar direitos, políticas)	mesmo link acima	US\$2 / US\$10 por milhão de tokens (promocional até 31/ago/2026; padrão US\$3/US\$15 depois)	Reservar para o chatbot "de verdade" (Interação 2), não para a classificação simples de tag
OpenAI API (linha "mini")	Alternativa equivalente de classificação/chat	https://openai.com/api/pricing/	Faixa de preço comparável à Anthropic Haiku/Sonnet	Avaliar apenas se a equipe já tiver relação contratual/prefêrência por este provedor;

				tecnicamente intercambiável
Classificador baseado em regras/dicionário (sem API externa)	Camada 1 de classificação, determinística, para a taxonomia fechada definida pela equipe	— (implementação própria)	US\$0	Nenhum dado sai da infraestrutura própria; ver recomendação na seção 7

6.4 Fora do escopo atual do PDF (mencionadas apenas como possibilidade futura)

API	Finalidade	Link oficial	Preços	Observação
Meta Graph API (Instagram)	Sincronizar automaticamente tag de origem a partir de posts/campanhas	https://developers.facebook.com/docs/instagram-platform	Gratuita para uso da API (custo real está na mídia paga das campanhas, não na API)	O PDF cita o exemplo "post → acesso → tag_origem" mas não exige automação via API; UTM manual em links de bio/anúncio resolve o mesmo requisito sem integração — recomendado para o MVP
WhatsApp Business Cloud API (Meta)	Canal de atendimento adicional	https://developers.facebook.com/docs/whatsapp/cloud-api (preços: https://developers.facebook.com/docs/whatsapp/pricing)	Brasil cobrando por mensagem (categorias utility/marketing/authentication); mensagens de serviço dentro da janela de 24h são gratuitas	Não pedido no PDF atual (o chatbot descrito é do site) — citado aqui só para não faltar, caso a equipe pergunte; não recomendar incluir nesta fase
Umami / Plausible (analytics privacy-friendly)	Métricas de tráfego padrão complementares ao modelo de eventos customizado	https://umami.is/ , https://plausible.io/	Umami: open-source, self-host gratuito. Plausible: self-host gratuito (AGPL) ou hospedado a	Alternativa a Google Analytics, evitando questões de transferência internacional de dados sob LGPD

			partir de ~US\$9/mês	
--	--	--	-------------------------	--

7. Opção mais barata recomendada (e por quê)

Território/mapa: IBGE + ViaCEP/BrasilAPI (grátis) + Nominatim para geocodificar equipamentos no cadastro (grátis, baixo volume) + MapLibre GL JS com tiles gratuitos para o mapa visual.

Justificativa: o PDF só pede geocodificação de fato para os *equipamentos cadastrados* (feita pelo admin, poucas vezes) e para localização manual da usuária (que pode ser resolvida por município/CEP, sem geocodificação alguma — a busca por proximidade usa o centróide do município ou o CEP resolvido via ViaCEP). Isso elimina completamente a necessidade de Google Maps/Mapbox pagos no MVP, sem perda de funcionalidade pedida.

Classificação de texto: classificador de regras/dicionário interno como camada 1, com Anthropic Claude Haiku 4.5 como fallback de baixa frequência para frases ambíguas.

Justificativa: a taxonomia do PDF é fechada e administrada pela equipe (não é um domínio aberto), o que torna um bom motor de regras (palavras-chave, sinônimos por tag/subtag) surpreendentemente eficaz e determinístico — e com custo zero e zero exposição de texto sensível a terceiros, o que também atende diretamente o princípio de minimização de dados exigido nas seções 14/24 do PDF. O LLM entra só quando a confiança das regras é baixa, minimizando tanto custo (Haiku é o modelo mais barato do mercado com boa capacidade de instrução, ~US\$1/US\$5 por milhão de tokens — cada classificação custa frações de centavo) quanto exposição de dado sensível.

Essa combinação é o melhor custo-benefício do conjunto: **custo de infraestrutura de dados geográficos e de classificação básica tende a US\$0/mês no MVP**, com gasto variável pequeno e proporcional ao uso apenas na parcela de IA usada como fallback.

8. Riscos, limitações e pontos de atenção

- Dependência de ativos ainda não entregues.** O PDF é explícito: identidade visual, taxonomia definitiva, conceitos de tipos de violência e base de equipamentos "serão fornecidos pela equipe". Sem esses insumos, o protótipo terá apenas andaime técnico (models, telas, fluxo) sem dado real — risco de cronograma que não é técnico, é de dependência externa.
- Erro de classificação por IA/regras não deve virar "diagnóstico".** O PDF exige isso explicitamente; é preciso desenho de UI que deixe clara a natureza de "orientação", e um mecanismo de correção pela usuária (não mencionado no PDF — lacuna, ver seção 9).

10. **Enviar texto de mulheres em situação de violência a uma API de terceiros (LLM) é sensível mesmo com minimização.** Recomenda-se validação jurídica (LGPD/DPA) antes de contratar qualquer provedor de IA para esse fluxo, mesmo usando apenas como fallback de baixo volume.
11. **Segurança do dispositivo/agressor.** Não é requisito explícito do PDF, mas é prática comum em produtos desse domínio (botão de saída rápida, descrição de histórico/notificações). Recomendo validar formalmente com a equipe se deve entrar no escopo.
12. **Cobertura incompleta dos 853 municípios.** Grande parte da UX depende do fallback "ainda não temos informações suficientes" funcionar bem — e da curadoria contínua da base, que é uma responsabilidade operacional (quem mantém?) não definida no PDF.
13. **Modelagem de eventos é a peça mais cara de errar.** O dashboard completo é explicitamente fora do escopo desta fase, mas a seção 20 do PDF já lista as perguntas analíticas que ele precisa responder no futuro — a tabela Journey::Event precisa ser desenhada certa desde já, porque corrigir modelagem de dados históricos depois é caro.
14. **Escolha de banco de dados.** SQLite/Solid stack é ótimo para começar, mas buscas geoespaciais em escala pedem PostgreSQL+PostGIS — decisão de infraestrutura que depende de orçamento/hospedagem ainda não confirmados.
15. **Papéis administrativos e fluxo de aprovação de parceiros não estão detalhados no PDF** — foram assumidos como hipótese (seção 3.7 e 9).
16. **Correção de escopo vinda por áudio/reunião, não por documento formal.** O ajuste da liderança (motivo e localização dentro do chat, não em telas separadas) foi recebido como transcrição de fala, o que deixa margem para imprecisão. Antes de construir a UI definitiva, recomendo validar um wireframe simples do chat (mostrando os cards de localização e resultado embutidos) diretamente com quem deu o feedback, para confirmar que a leitura deste parecer está alinhada com a intenção real — evita retrabalho de front-end depois.

9. Lacunas do PDF — dúvidas e hipóteses antes de implementar

As seguintes perguntas **não têm resposta no PDF** e foram tratadas como hipóteses neste parecer; a implementação real deveria confirmá-las antes de começar:

17. ~O motivo e a localização são coletados em telas de formulário separadas antes do chat, ou dentro da própria conversa?~ **Resolvido pela liderança:** dentro da própria conversa (Entrada 2) — aplicado nas seções 1, 2 e 3.2.1. Falta apenas confirmar o wireframe (ver risco 9 na seção 8).
18. A base de equipamentos/mapeamento territorial já existe em algum formato (planilha, outro sistema)? Qual volume e formato para migração?
19. A taxonomia definitiva (tags/subtags) já existe em documento à parte, ou este protótipo deve propor uma taxonomia inicial de rascunho? *A liderança já deu exemplos concretos de conteúdo/origem (tipos de violência, Lei Maria da Penha, delegacia especializada, casa de

abrigo) — vale pedir a lista completa de tags/subtags de origem usada hoje no Instagram, para não recriá-la.*

20. Quem administra a taxonomia e o conteúdo no dia a dia — a própria equipe Maria Mineira (não-técnica, via painel) ou o time de desenvolvimento (via seed/código)? *(Assumido: painel administrativo, hipótese.)*
 21. Qual o volume esperado de acessos/mês? Há campanhas de mídia paga que possam gerar picos relevantes de tráfego? Isso muda diretamente a recomendação de banco de dados e o dimensionamento de custo de IA.
 22. Existe orçamento definido para APIs pagas (IA, mapas), ou a diretriz é buscar soluções gratuitas sempre que possível? *(Assumido: priorizar gratuito, dado o caráter público/social do projeto — ver seção 7.)*
 23. O chatbot deve rodar sobre um LLM de terceiros específico (Anthropic/OpenAI) ou a equipe já tem contrato/preferência com algum provedor?
 24. Há exigência legal de retenção mínima de dados (ex.: auditoria de política pública) que possa conflitar com a minimização pedida nas seções 14/24 do PDF?
 25. Quais e quantos são os perfis administrativos, e como funciona o fluxo de aprovação de parceiros na "Rede Maria Mineira"? *(Assumidos 4 papéis genéricos como hipótese — seção 3.7.)*
 26. Hospedagem/infraestrutura: já há provedor definido? O ambiente atual tem o MCP do Railway configurado, o que sugere essa possibilidade, mas não é confirmação.
 27. O domínio próprio citado no PDF já está registrado e sob gestão de quem para o deploy final?
 28. O "botão de saída rápida"/modo discreto (prática comum no setor, não citada no PDF) deve entrar no escopo deste protótipo?
 29. Qual precisão geográfica é aceitável no MVP — bairro/CEP/município bastam, ou é necessário raio em metros por coordenada exata?
 30. O escopo é permanentemente Minas Gerais (853 municípios) ou há intenção futura de expandir para outros estados (o que mudaria a modelagem de Territorial::Municipality, hoje assumida com state fixo "MG")?
 31. A liderança mencionou que o rastreamento de origem funciona "se tiver ativado ou não" — isso significa (a) que é uma configuração que a equipe liga/desliga por campanha, (b) que depende simplesmente de o link carregar ou não um parâmetro de rastreamento (UTM), ou (c) outra coisa? *Hipótese assumida neste parecer: opção (b), mais simples e sem necessidade de painel de configuração.*
 32. Dentro do chat, quando a pessoa muda de assunto depois do resultado (estágio "livre"), cada novo tópico deve gerar um tag_chat novo automaticamente, ou deve haver algum limite/agrupamento (ex.: só o assunto predominante da sessão, como sugere a seção 12 do PDF: "o assunto predominante dessa segunda interação")? *Hipótese assumida: registrar todos os tópicos detectados como eventos, deixando o "predominante" para a camada analítica calcular depois — mais flexível para o diagnóstico territorial.*
-

Próximos passos sugeridos

33. Confirmar com a liderança, num wireframe simples do chat (motivo → botão de localização → cards de resultado, tudo numa tela só), que a leitura deste parecer bateu com a intenção real — é a validação mais barata antes de qualquer linha de código de front-end.
34. Validar as demais hipóteses da seção 9 com a equipe Maria Mineira (especialmente 1, 2, 4, 5, 6 e 9 — bloqueiam decisões de custo/infra).
35. Com as respostas, transformar este parecer em plano de implementação técnico (scaffold Rails 8.1, migrations reais, seeds de taxonomia e municípios via IBGE).
36. Publicar este parecer como documento compartilhável (artifact) para facilitar a validação com stakeholders não-técnicos, se desejado.